MLE-AI-201 MACHINE LEARNING · FINAL PROJECT

A Decision Tree and a Pegasos SVM from Scratch

Predicting Baku property prices from bina.az listings

Alimammadova, Nigar
Dilavarli, Farid
Gojayev, Nihad
Karimova, Ulkar

Speaker: Nigar Alimammadova · Introduction

PROBLEM AND DATASET

One dataset, two prediction tasks

100,775
raw rows, 51 columns, scraped from bina.az

64,248
distinct listings after cleaning

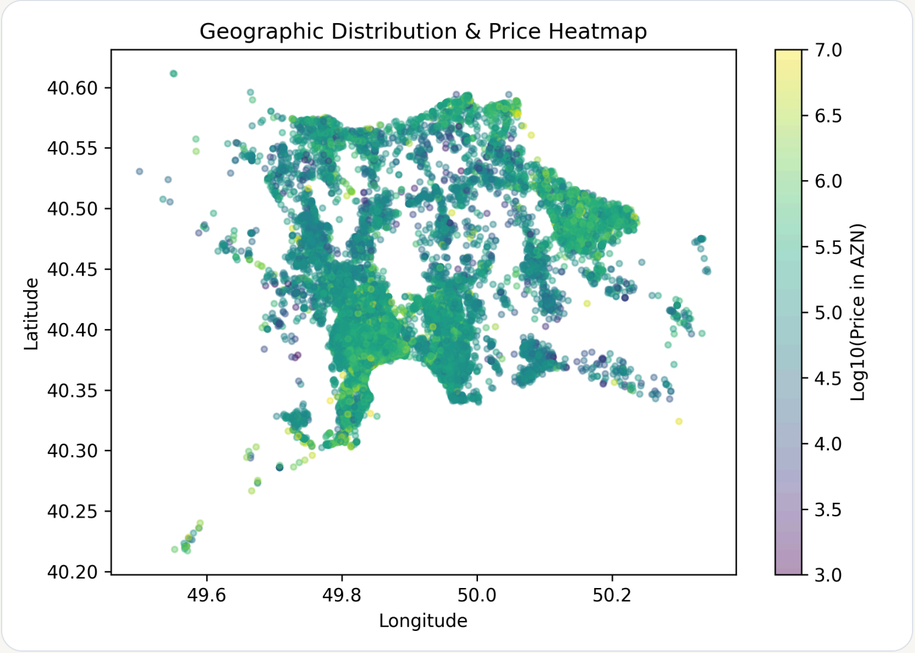
99.7%
of listings are in Baku; prices in AZN

Task A · Regression

Predict the price. We model log(price), because prices run from 1,000 to 10 million AZN and the log is close to symmetric.

Task B · Classification

Premium or standard: above or below the median training price, 210,000 AZN. The test set never sets this threshold.



Listings on the map, coloured by price: the centre and the coast cost most.

CLEANING AND FEATURES

A raw scrape, turned into a modelling table

Problem	What we did
36% of rows are repeat scrapes	Kept one row per listing URL, the latest
Floor as text, “7 / 9”	Split into floor and floors in the building
Land in sot, flats in m²	Converted sot to m² (1 sot = 100 m²)
Extreme values	Price 1,000–10M AZN; area 15–3,000 m² (land up to 50,000)
Missing values	Constants: 0 for numbers, “Unknown” for text

165 features: 7 numeric and 158 one-hot columns. Split once, stratified, 70 / 15 / 15: 44,976 train, 9,636 validation, 9,636 test.

Leakage removed

unit_price × area gives the price within 0.3%.

total_price equals the price in 99.99% of rows.

Ids, owner names, address and post-publication columns (views, vip) are never used.

The hidden leak: with repeats kept, **48% of test rows** were also in training.

MODEL 1 · DECISION TREE

A decision tree in NumPy, for both tasks

Impurity

$$Gini = 1 - \Sigma\, p^2$$

$$H = -\Sigma\, p \cdot \log_2 p$$

$$MSE = \text{mean}\,(y - \bar{y})^2$$

Gini or entropy for the tier, variance for the price.

Split search

Sort each feature; candidate thresholds are midpoints between neighbouring values.

$$\Delta I = I(\text{parent}) - \Sigma\,(N_c/N) \cdot I(c)$$

Take the split with the largest decrease, then recurse.

Stopping rules

`max_depth`
`min_samples_split`
`min_samples_leaf`
`min_impurity_decrease`

A leaf predicts the majority class (with class shares as probabilities) or the mean price.

depth 15 · leaf ≥ 20 · Gini
chosen on the validation split

R^2 0.781 · F1 0.912
test split; scikit-learn: 0.782 and 0.912

99.9%
same tier calls as scikit-learn’s tree

Speaker: Nihad Gojayev · Decision tree

MODEL 2 · PEGASOS SVM

A soft-margin SVM without a QP solver

Hinge-loss primal

$$J(w,b) = \lambda/2 \cdot \|w\|^2 + 1/n \cdot \Sigma \max(0,\ 1 - y_i(w \cdot x_i + b))$$

Pegasos update, step t

$$\eta = 1/(\lambda t)$$

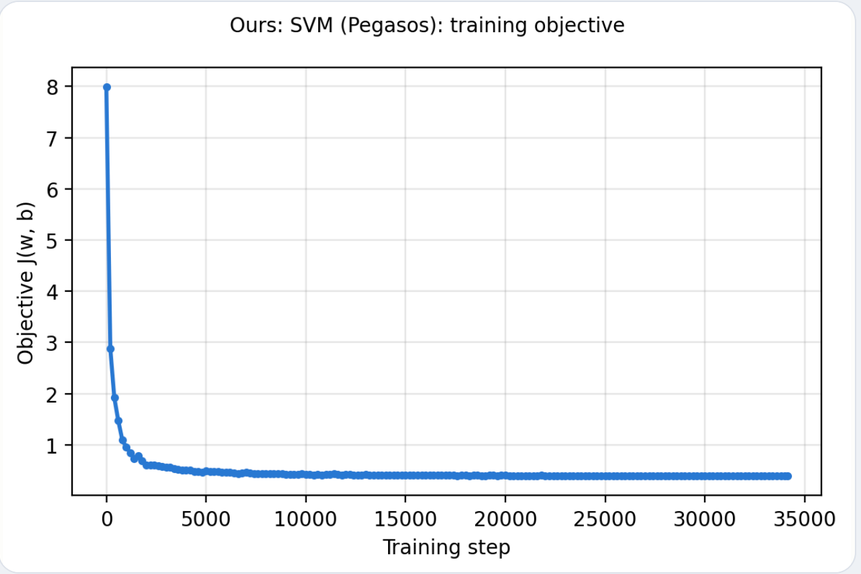
$$w \leftarrow (1 - \eta\lambda) \cdot w + \eta \cdot y \cdot x \text{ if } y(w \cdot x) < 1$$

$$w \leftarrow (1 - \eta\lambda) \cdot w \text{ otherwise}$$

Mini-batches of 32, bias folded into w, features standardised on training rows.

Non-linear: random Fourier features

$z(x) = \sqrt{(2/D)} \cdot \cos(Wx + b)$ approximates an RBF kernel; the same linear solver runs on $z(x)$.



$\lambda = 0.001$, chosen on validation. Margin $2/\|w\| = 0.60$; 20,957 of 54,612 training points are support vectors.

Speaker: Nigar Alimammadova · Pegasos SVM

EXPERIMENTAL SETUP

Same data, same rules, for our models and the library’s

Metrics, written by us

Price: RMSE, MAE, R^2 on log price. Tier: precision, recall, F1, ROC-AUC, confusion matrix. Checked against scikit-learn to 12 digits.

Matched baselines

DecisionTreeRegressor and Classifier with our stopping rules; SGDClassifier with hinge loss and $\alpha = \lambda$, scaled inside a Pipeline.

No peeking at the test set

Threshold and scaling from training rows only. Settings chosen on validation. 5-fold cross-validation for mean ± std; the test split is scored once.

One command

`python -m src.run_all` rebuilds every number, table and figure in about 5 minutes. Seed 42; 111 automated tests.

Speaker: Farid Dilavarli · Evaluation and benchmark

RESULTS

Both our models match scikit-learn

Model (test split)	A: R^2	B: F1	B: AUC	CV F1
Ours: tree	0.781	0.912	0.969	0.910 ± 0.002
sklearn: tree	0.782	0.912	0.969	0.909 ± 0.003
Ours: SVM	—	0.850	0.925	0.842 ± 0.007
sklearn: SGD	—	0.852	0.927	0.846 ± 0.005

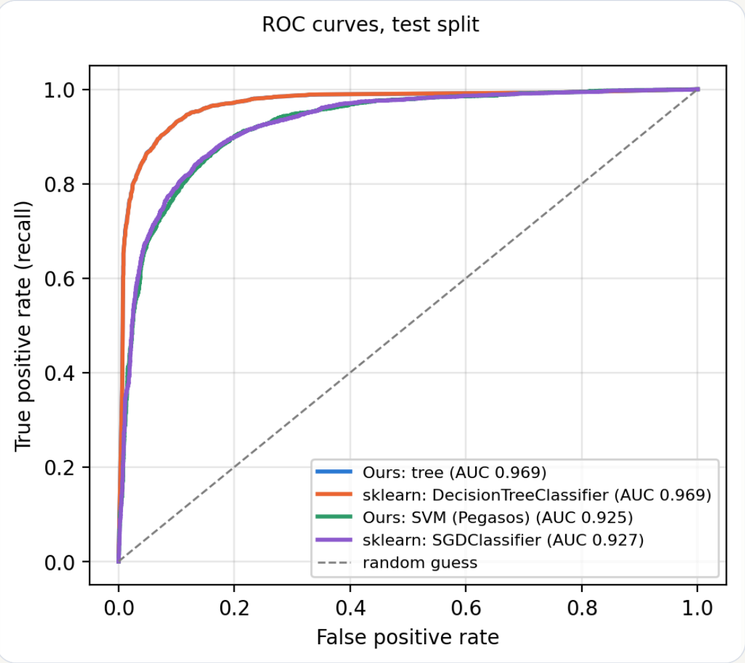
99.9%

identical tier calls, our tree vs scikit-learn's

95.7%

identical tier calls, our SVM vs SGDClassifier

A = price on log scale, B = premium tier; CV = 5-fold mean ± std. — : the SVM does only Task B.

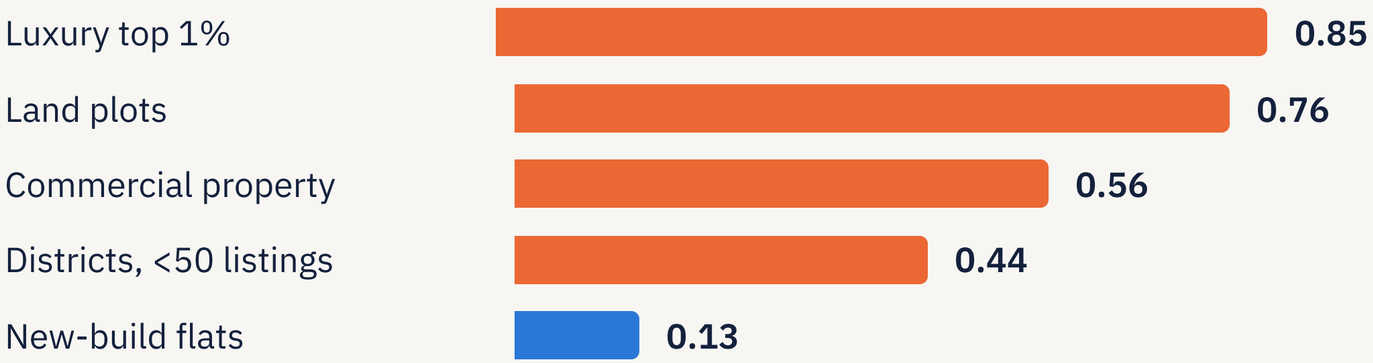


ROC curves, test split: the tree ranks better than any linear model.

ERROR ANALYSIS AND LIMITATIONS

Rare and expensive listings are the hard part

Average miss on log price, test split (our tree)



0.13 ≈ a 14% miss; 0.85 ≈ off by a factor of 2.3. Few examples, and prices set by things the table lacks.

Tier task: the boundary

Mistakes pile up near 210,000 AZN: our tree is wrong on 9–11% of listings in the middle price bands, on 0–3% at either end.

Honest limitations

- Asking prices, not sale prices
- One model for seven property types
- Missing numbers set to 0
- Long tail of land area hurts the linear SVM
- Data from early 2025, almost all Baku

Speaker: Farid Dilavarli · Error analysis

CONTRIBUTIONS AND NEXT STEPS

Who did what, and what we would do next

Ulkar Karimova

Data cleaning, leakage removal, features, stratified split, EDA figures

Nihad Gojayev

Decision tree: impurity measures, split search, growth and stopping rules

Nigar Alimammadova

Pegasos SVM, random Fourier features, SVM studies; the repository

Farid Dilavarli

Metrics, benchmark, one-command pipeline, tuning, error analysis, report

Next steps

- A separate model per property type
- A random forest built from our own trees
- Log of area, real missing-value flags
- More cities and newer listings